

Documentație Proiect TAP

Magazin Online de Electronice și Electrocasnice

Barbul Laurențiu, Opreța Mario

10 iunie 2026

Contents

1	Prezentare generală	3
1.1	Descrierea proiectului	3
1.2	Tehnologii folosite	3
1.3	Arhitectura generală	3
2	Cerințe de sistem	4
3	Ghid de instalare pas cu pas	5
3.1	1. Clonarea proiectului	5
3.2	2. Instalarea dependențelor	5
3.3	3. Configurarea bazei de date PostgreSQL	5
3.4	4. Configurarea variabilelor de mediu	5
3.5	5. Generarea clientului Prisma, rularea migrărilor și seed	5
3.6	6. Pornirea backend-ului	6
3.7	7. Pornirea frontend-ului	6
3.8	8. Comenzi de verificare	6
4	Structura proiectului	6
4.1	Fișiere și foldere importante	7
5	Modelul bazei de date	8
5.1	Entități principale	8
5.2	Fragment relevant: schema Prisma	8
5.3	Integritate și constrângeri	9
6	Backend	9
6.1	Configurarea aplicației Express	9
6.2	Securitate: JWT și rol admin	9
6.3	Autentificare și cont utilizator	10
6.4	Catalog produse	10
6.5	Coș și checkout	11
6.6	Funcționalități admin	11
7	Frontend	12
7.1	Rutare și layout	12
7.2	Comunicarea cu API-ul	12

7.3	Autentificare în frontend	12
7.4	Catalogul de produse	13
7.5	Coș și checkout în frontend	13
7.6	Panoul de administrare	14
8	Referință API	14
9	Testare și stabilizare	15
9.1	Backend	15
9.2	Frontend	15
9.3	Recomandări pentru extinderea testării	15
10	Integrarea cerințelor din syllabus	16
10.1	Cerințe minime din syllabus	16
10.2	Organizarea echipei conform syllabus	16
11	Indicații centralizate pentru imagini	17
12	Concluzii	17

1 Prezentare generală

1.1 Descrierea proiectului

Proiectul reprezintă o aplicație full stack de tip magazin online pentru produse electronice și electrocasnice. Aplicația permite utilizatorilor să creeze cont, să își verifice adresa de email, să se autentifice, să navigheze prin catalogul de produse, să filtreze și să sorteze produsele, să adauge articole în coș, să finalizeze o comandă simulată și să consulte istoricul comenzilor.

Pentru administratori, aplicația oferă funcționalități de administrare: gestionarea produselor, categoriilor, comenzilor și utilizatorilor. Backend-ul expune un API REST protejat prin JWT, iar persistența este realizată într-o bază de date relațională PostgreSQL, administrată prin Prisma ORM.

Proiectul corespunde temei din Syllabus.pdf: "Magazin Online de Electronice și Electrocasnice (React + Node.js + PostgreSQL)". Funcționalitățile implementate acoperă cerințele minime din syllabus: register/login, listare și detalii produse, coș de cumpărături, istoric comenzi și CRUD admin pentru produse.

Indicație imagine 1: inserați aici un screenshot al paginii principale a aplicației, cu header-ul, bara de căutare și zona de produse/oferte vizibile.

1.2 Tehnologii folosite

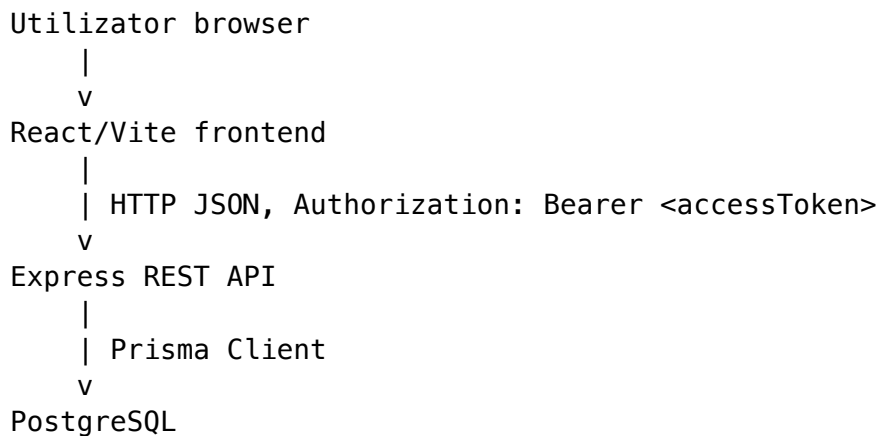
Nivel	Tehnologii	Rol în proiect
Frontend	React 19, TypeScript, Vite, React Router, Redux Toolkit, React Query, React Hook Form, Zod	Interfață web, rutare, stare de autentificare, formulare, validare și comunicare cu API-ul
Backend	Node.js, Express 5, TypeScript, Zod, JWT, bcrypt, Helmet, CORS, Morgan, Rate Limit	API REST, securitate, autentificare, validare input, reguli business
Bază de date	PostgreSQL, Prisma ORM, Prisma Migrate	Persistență relațională, modele, relații, migrări și seed
Email	Nodemailer	Trimitere email pentru verificare cont și resetare parolă
Testare	Vitest, Supertest, Testing Library	Smoke tests backend și test frontend pentru pagina Home
Tooling	npm, tsx, ESLint, TypeScript compiler	Rulare locală, build, verificări și dezvoltare

1.3 Arhitectura generală

Aplicația este organizată în trei zone principale:

1. **Frontend (client/)** - aplicație React servită de Vite. Pagina comunică cu backend-ul prin Axios și folosește React Query pentru date server-side.
2. **Backend (server/)** - aplicație Express care expune endpoint-uri REST sub prefixul /api.
3. **Bază de date (database/prisma/)** - schema Prisma, migrările și seed-ul inițial pentru PostgreSQL.

Fluxul general al datelor este:



În modul de dezvoltare, Vite proxy-uește cererile /api către backend-ul local de pe `http://localhost:4000`, conform fișierului `client/vite.config.ts`.

2 Cerințe de sistem

Pentru instalarea și rularea proiectului sunt necesare următoarele aplicații și dependențe:

Cerință	Versiune recomandată	Observații
Node.js	20+ sau 24+	Proiectul a fost inspectat cu Node instalat prin Homebrew; pachetele folosesc TypeScript modern
npm	inclus cu Node.js	Folosit pentru instalarea dependențelor și rularea scripturilor
PostgreSQL	14+	Necesită o bază de date locală, de exemplu proiecttap
Git Editor	orice versiune recentă VS Code recomandat	Pentru clonarea repository-ului Util pentru lucrul cu TypeScript și Prisma
Prisma CLI	instalat prin npm	Scripturile folosesc Prisma din dependențele proiectului
Cont SMTP	opțional pentru demo complet email	Necesar pentru trimiterea reală a emailurilor de verificare/resetare

Dependențele principale sunt definite în trei fișiere:

- `package.json` la rădăcină: scripturi agregate și Prisma.
- `server/package.json`: Express, Prisma Client, JWT, bcrypt, Nodemailer, Zod, Vitest, Supertest.
- `client/package.json`: React, Vite, React Router, Redux Toolkit, React Query, React Hook Form, Zod.

3 Ghid de instalare pas cu pas

3.1 1. Clonarea proiectului

```
git clone https://github.com/OpritaMario26/proiectTAP.git  
cd proiectTAP
```

Dacă proiectul este deja local, deschideți folderul rădăcină proiectTAP în VS Code.

3.2 2. Instalarea dependențelor

Pentru că proiectul are pachete separate pentru rădăcină, backend și frontend, rulați:

```
npm install  
npm --prefix server install  
npm --prefix client install
```

3.3 3. Configurarea bazei de date PostgreSQL

Creați o bază de date locală:

```
createdb proiecttap
```

Dacă folosiți un alt utilizator sau altă parolă, actualizați corespunzător variabila DATABASE_URL din fișierul .env al serverului.

3.4 4. Configurarea variabilelor de mediu

Copiați exemplul de configurare:

```
cp server/.env.example server/.env
```

Completați server/.env cu valori reale sau de dezvoltare:

```
PORT=4000  
DATABASE_URL="postgresql://postgres:postgres@localhost:5432/proiecttap?schema=public"  
JWT_ACCESS_SECRET="change-me-access-secret-minim-16-caractere"  
JWT_REFRESH_SECRET="change-me-refresh-secret-minim-16-caractere"  
CORS_ORIGIN="http://localhost:5173"  
SMTP_HOST="smtp.gmail.com"  
SMTP_PORT=587  
SMTP_USER="utilizator-smtp"  
SMTP_PASS="parola-smtp"  
SMTP_FROM="noreply@ecommerce.com"
```

Observație importantă: codul backend validează SMTP_USER și SMTP_PASS ca valori obligatorii. Pentru un demo local fără trimitere reală de email, se pot completa valori de test, însă emailurile nu vor fi livrate. În acest caz, endpoint-ul de register poate returna tokenul de verificare în răspuns dacă trimiterea emailului eșuează.

3.5 5. Generarea clientului Prisma, rularea migrărilor și seed

Din rădăcina proiectului:

```
npm run db:generate
npm run db:migrate
npm run db:seed
```

Aceste comenzi generează Prisma Client, creează tabelele în PostgreSQL și inserează date inițiale: categorii, produse și un utilizator admin de test.

3.6 6. Pornirea backend-ului

Într-un terminal:

```
npm run dev:server
```

Backend-ul pornește implicit pe `http://localhost:4000`. Verificare rapidă:

```
curl http://localhost:4000/api/health
```

Răspuns așteptat:

```
{
  "status": "ok",
  "service": "backend",
  "timestamp": "..."
```

3.7 7. Pornirea frontend-ului

Într-un al doilea terminal:

```
npm run dev:client
```

Aplicația frontend rulează de regulă pe `http://localhost:5173`.

Indicație imagine 2: inserați aici un screenshot cu aplicația rulând în browser la `http://localhost:5173`, ideal cu pagina de produse deschisă.

3.8 8. Comenzi de verificare

Pentru build complet:

```
npm run build
```

Pentru testele backend:

```
npm --prefix server run test
```

Pentru testele frontend:

```
npm --prefix client run test
```

4 Structura proiectului

Structura principală este:

```
proiectTAP/
├── API_ENDPOINTS.md
└── README.md
```

```

├── plan.md
├── Syllabus.pdf
├── client/
│   ├── package.json
│   ├── vite.config.ts
│   └── src/
│       ├── App.tsx
│       ├── api/http.ts
│       ├── pages/
│       ├── store/
│       └── types/
├── database/
│   └── prisma/
│       ├── schema.prisma
│       ├── seed.ts
│       └── migrations/
├── server/
│   ├── package.json
│   └── src/
│       ├── app.ts
│       ├── server.ts
│       ├── config/
│       ├── lib/
│       ├── middleware/
│       └── routes/
└── Prezentare/

```

4.1 Fișiere și foldere importante

Zonă	Rol
API_ENDPOINTS.md	Documentație tehnică a endpoint-urilor backend
Syllabus.pdf	Cerințele academice ale proiectului
client/src/App.tsx	Rutarea principală și layout-ul aplicației React
client/src/api/http.ts	Instanța Axios folosită pentru apeluri HTTP
client/src/pages/	Pagini pentru Home, login, catalog, coș, checkout, comenzi și admin
client/src/store/authSlice.ts	Starea locală pentru sesiunea utilizatorului
server/src/app.ts	Configurarea Express, middleware și rutare API
server/src/routes/	Modulele backend pentru auth, produse, categorii, coș, comenzi, utilizatori
server/src/lib/auth.ts	Middleware de autentificare JWT și autorizare admin
server/src/config/env.ts	Validarea variabilelor de mediu cu Zod
database/prisma/schema.prisma	Modelul relațional al bazei de date
database/prisma/seed.ts	Date inițiale pentru categorii, produse și admin

5 Modelul bazei de date

Schema Prisma definește tabelele cerute în syllabus: USERS, CATEGORIES, PRODUCTS, CART, ORDERS, ORDER_ITEMS.

5.1 Entități principale

Tabel	Scop	Relații
USERS	Conturi de utilizator, roluri, hash parole, tokenuri refresh/reset/verificare	Un utilizator are articole în coș și comenzi
CATEGORIES	Categorii de produse, cu slug unic	O categorie are mai multe produse
PRODUCTS	Produse cu preț, stoc, imagine, brand și categorie	Produsul aparține unei categorii și poate apărea în coș/comenzi
CART	Produsele salvate în coșul unui utilizator	Legătură utilizator-produs, unică pe perechea (userId, productId)
ORDERS	Comenzi plasate, cu status și adresă livrare	O comandă aparține unui utilizator și are mai multe poziții
ORDER_ITEMS	Liniiile unei comenzi, cu preț unitar istoric	Leagă comanda de produs

5.2 Fragment relevant: schema Prisma

```
model Product {
  id          Int      @id @default(autoincrement())
  categoryId  Int      @map("category_id")
  name        String
  slug        String   @unique
  description String
  price       Decimal  @db.Decimal(10, 2)
  stock       Int
  imageUrl    String   @map("image_url")
  brand       String

  category    Category @relation(fields: [categoryId], references: [id], onDelete: Restrict)
  cartItems   CartItem[]
  orderItems  OrderItem[]

  @@index([categoryId], map: "idx_products_category_id")
  @@index([brand], map: "idx_products_brand")
  @@index([price], map: "idx_products_price")
  @@map("PRODUCTS")
}
```

Explicație: modelul Product descrie produsul comercial afișat în catalog. Relația cu Category impune existența unei categorii valide, iar indecșii pe categorie, brand și preț ajută operațiile de filtrare și sortare din catalog.

5.3 Integritate și constrângeri

- email și slug sunt unice pentru a evita conturi/URL-uri duplicate.
- Coșul are constrângere unică pe (userId, productId), astfel încât un produs apare o singură dată în coș, cu o cantitate actualizabilă.
- Ștergerea unei categorii este blocată dacă există produse asociate.
- Ștergerea unui produs este blocată dacă acesta apare în comenzi active.
- Comenzile păstrează unitPrice în ORDER_ITEMS, pentru istoric financiar corect chiar dacă prețul produsului se schimbă ulterior.

Indicație imagine 3: inserați aici o diagramă ERD a bazei de date cu tabelele USERS, CATEGORIES, PRODUCTS, CART, ORDERS, ORDER_ITEMS și relațiile dintre ele.

6 Backend

6.1 Configurarea aplicației Express

Backend-ul este construit ca API REST modular. Fișierul server/src/app.ts configurează middleware-urile globale și conectează rutele.

```
app.use(helmet());
app.use(cors({ origin: env.CORS_ORIGIN, credentials: true }));
app.use(morgan('dev'));
app.use(express.json());
app.use(cookieParser());

app.use('/api/health', healthRouter);
app.use('/api/auth', authRouter);
app.use('/api/products', productsRouter);
app.use('/api/categories', categoriesRouter);
app.use('/api/cart', cartRouter);
app.use('/api/orders', ordersRouter);
app.use('/api/users', usersRouter);
```

Explicație: helmet adaugă headere de securitate, cors permite frontend-ului să comunice cu API-ul, morgan loghează cererile, iar express.json() parsează body-urile JSON. Rutele sunt separate pe domenii funcționale, ceea ce face aplicația mai ușor de întreținut.

6.2 Securitate: JWT și rol admin

Autentificarea folosește access token JWT cu durată de 15 minute și refresh token cu durată de 7 zile. Middleware-ul authenticate verifică headerul Authorization, validează tokenul și încarcă utilizatorul din baza de date.

```
const authHeader = req.headers.authorization;
if (!authHeader || !authHeader.startsWith('Bearer ')) {
  res.status(401).json({ message: 'Authorization header missing or invalid' });
  return;
}

const token = authHeader.substring(7);
const decoded = jwt.verify(token, env.JWT_ACCESS_SECRET) as any;
```

```
const user = await prisma.user.findUnique({ where: { id: decoded.sub } });

req.user = { id: user.id, email: user.email, role: user.role };
next();
```

Explicație: middleware-ul separă autentificarea de logica rutelor. Rutele pentru coș, comenzi și profil pot presupune că req.user există după trecerea prin acest middleware. Pentru funcțiile de admin, middleware-ul requireAdmin verifică rolul ADMIN și returnează 403 pentru utilizatori obișnuiți.

6.3 Autentificare și cont utilizator

Modulul auth.ts include:

- POST /api/auth/register - creare cont, hash parolă cu bcrypt, token verificare email.
- GET /api/auth/verify-email - confirmare adresă email.
- POST /api/auth/login - verificare credentiale, emitere access token și refresh token.
- POST /api/auth/refresh - generare access token nou.
- POST /api/auth/request-reset și POST /api/auth/reset-password - resetare parolă.
- GET/PUT /api/auth/profile - citire și actualizare profil.
- PUT /api/auth/change-password - schimbare parolă autentificată.

Validarea se face cu Zod. De exemplu, parola trebuie să aibă cel puțin 8 caractere, o literă mică, o literă mare și o cifră.

6.4 Catalog produse

Endpoint-ul GET /api/products permite căutare, filtrare, paginare și sortare.

```
const whereClause = {
  ...(search
    ? {
      OR: [
        { name: { contains: search, mode: 'insensitive' as const } },
        { brand: { contains: search, mode: 'insensitive' as const } },
        { category: { name: { contains: search, mode: 'insensitive' as const } } } ],
      }
    : {}),
  ...(categoryId ? { categoryId: Number(categoryId) } : {}),
};

const [total, products] = await Promise.all([
  prisma.product.count({ where: whereClause }),
  prisma.product.findMany({
    where: whereClause,
    include: { category: true },
    orderBy,
    skip: (page - 1) * pageSize,
    take: pageSize,
  }),
]);
```

Explicație: API-ul construiește dinamic filtrul Prisma în funcție de query parameters. `Promise.all` calculează în paralel totalul și lista paginată, ceea ce permite frontend-ului să afișeze atât produsele, cât și metadatele de paginare.

6.5 Coș și checkout

Coșul este disponibil doar pentru utilizatori autentificați. Endpoint-urile permit citirea coșului, adăugarea unui produs, actualizarea cantității și ștergerea unui produs. Cantitatea este limitată la 99, iar stocul este verificat înainte de inserare sau actualizare.

Checkout-ul creează o comandă din articolele din coș. Operația este tranzacțională: se creează comanda, se inserează liniile comenzii, se scade stocul și se golește coșul.

```
const result = await prisma.$transaction(async (tx) => {
  const order = await tx.order.create({
    data: {
      userId,
      totalAmount,
      shippingAddress,
      orderItems: { create: orderItems },
    },
    include: {
      orderItems: { include: { product: true } },
    },
  });

  for (const item of cartItems) {
    await tx.product.update({
      where: { id: item.productId },
      data: { stock: { decrement: item.quantity } },
    });
  }

  await tx.cartItem.deleteMany({ where: { userId } });
  return order;
});
```

Explicație: tranzacția garantează că checkout-ul nu rămâne într-o stare parțială. Dacă scăderea stocului sau crearea comenzii eșuează, toate operațiile sunt anulate.

Indicație imagine 4: inserați aici un screenshot al coșului de cumpărături cu cel puțin două produse și totalul comenzii vizibil.

6.6 Funcționalități admin

Rutele admin sunt protejate prin `authenticate` și `requireAdmin`:

- produse: creare, editare, ștergere;
- categorii: creare, editare, ștergere;
- comenzi: vizualizare toate comenzile și modificare status;
- utilizatori: listare, schimbare rol, ștergere cu reguli de protecție.

Aceste funcționalități acoperă Modulul 5 din syllabus: roluri USER/ADMIN, protecție endpoint-uri și CRUD produse/categorii.

7 Frontend

7.1 Rutare și layout

Frontend-ul este organizat în pagini React. `App.tsx` definește meniul principal și rutele aplicației.

```
function AdminRoute({ children }: { children: ReactElement }) {
  const user = useAppSelector((state) => state.auth.user);
  if (!user) {
    return <Navigate to='/login' replace />;
  }
  if (user.role !== 'ADMIN') {
    return <Navigate to='/' replace />;
  }
  return children;
}
```

Explicație: componenta `AdminRoute` protejează paginile administrative în interfață. Dacă utilizatorul nu este autentificat, este trimis la login; dacă nu are rol admin, este redirecționat la pagina principală. Protecția reală rămâne totuși pe backend, unde cererile sunt validate cu JWT.

7.2 Comunicarea cu API-ul

Instanța `Axios` este definită în `client/src/api/http.ts`:

```
export const http = axios.create({
  baseURL: import.meta.env.VITE_API_URL ?? '/api',
});
```

În dezvoltare, `/api` este proxy-uit de Vite către `http://localhost:4000`, astfel încât frontend-ul poate apela `http.get('/products')` fără să hardcodeze hostul backend-ului.

7.3 Autentificare în frontend

Pagina de login folosește `React Hook Form` și `Zod` pentru validare, apoi salvează sesiunea în `Redux`.

```
const onSubmit = async (data: LoginForm) => {
  try {
    const response = await http.post<LoginResponse>('/auth/login', data);
    dispatch(setSession(response.data));
    navigate('/');
  } catch (error) {
    setError('root', { message: 'Autentificare esuata. Incearca din nou.' });
  }
};
```

Explicație: după autentificare, backend-ul returnează `access token`, `refresh token` și datele utilizatorului. Acestea sunt salvate în `authSlice`, iar restul aplicației poate verifica dacă utilizatorul este logat și ce rol are.

Indicație imagine 5: inserați aici un screenshot al formularului de login, ideal cu un exemplu de mesaj de validare sau eroare.

7.4 Catalogul de produse

Pagina `ProductsPage.tsx` folosește React Query pentru încărcarea produselor și categoriilor. Filtrele sunt păstrate în query parameters, ceea ce permite linkuri partajabile pentru căutări.

```
const productsQuery = useQuery({
  queryKey: ['products', search, sort, categoryId],
  queryFn: () => fetchProducts(search, sort, categoryId),
});

const addToCartMutation = useMutation({
  mutationFn: async (productId: number) => {
    if (!accessToken) {
      navigate('/login');
      return;
    }

    await http.post(
      '/cart',
      { productId, quantity: 1 },
      { headers: { Authorization: `Bearer ${accessToken}` } },
    );
  },
});
```

Explicație: queryKey include filtrul curent, deci React Query reîncarcă automat datele când se schimbă căutarea, sortarea sau categoria. Mutarea `addToCartMutation` verifică sesiunea și trimite tokenul JWT în header.

Indicație imagine 6: inserați aici un screenshot al paginii de produse cu filtrele de categorie/sortare și carduri de produse vizibile.

7.5 Coș și checkout în frontend

Pagina coșului încarcă articolele curente și permite incrementarea, decrementarea sau ștergerea unui produs. După fiecare mutație, query-ul `cart` este invalidat pentru reîmprospătarea datelor.

```
const updateMutation = useMutation({
  mutationFn: async ({ productId, quantity }) => {
    await http.put(
      `/cart/${productId}`,
      { quantity },
      { headers: { Authorization: `Bearer ${accessToken}` } },
    );
  },
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ['cart'] });
  },
});
```

Explicație: frontend-ul trimite doar cantitatea nouă, iar backend-ul decide dacă produsul rămâne în coș sau este șters atunci când cantitatea ajunge la zero. Această separare păstrează regulile de business pe server.

Pagina de checkout trimite adresa de livrare către POST /api/orders, iar după succes invalidează coșul și istoricul comenzilor.

Indicație imagine 7: inserați aici un screenshot al paginii de checkout cu formularul de adresă și butonul de finalizare comandă.

7.6 Panoul de administrare

Pagina AdminProductsPage.tsx permite adăugarea, editarea și ștergerea produselor. Formularul este validat cu Zod, iar operațiile sunt trimise către endpoint-urile admin.

```
const productSchema = z.object({
  categoryId: z.number().int().positive('Selecteaza categoria'),
  name: z.string().min(2, 'Numele este obligatoriu'),
  slug: z.string().regex(/^[a-z0-9-]+$/, 'Slug invalid'),
  price: z.number().positive('Pret invalid'),
  stock: z.number().int().min(0, 'Stoc invalid'),
  imageUrl: z.string().url('URL imagine invalid'),
  brand: z.string().min(1, 'Brand obligatoriu'),
});
```

Explicație: validarea pe client îmbunătățește experiența utilizatorului, dar aceleași reguli critice sunt replicate și pe backend. Astfel, aplicația rămâne protejată și dacă cineva trimite cereri direct către API.

Indicație imagine 8: inserați aici un screenshot al panoului Admin Produse, cu formularul de produs și tabelul produselor existente.

8 Referință API

Detaliile complete sunt documentate în API_ENDPOINTS.md. Rezumatul endpoint-urilor principale este:

Domeniu	Endpoint-uri principale	Autentificare
Auth	POST /api/auth/register, POST /api/auth/login, POST /api/auth/refresh, GET /api/auth/profile	parțial
Products	GET /api/products, GET /api/products/:slug, POST/PUT/DELETE /api/products	admin pentru modificări
Categories	GET /api/categories, POST/PUT/DELETE /api/categories	admin pentru modificări
Cart	GET/POST/PUT/DELETE /api/cart	utilizator autentificat

Domeniu	Endpoint-uri principale	Autentificare
Orders	GET /api/orders, POST /api/orders, PUT /api/orders/:id/status	utilizator autentificat; admin pentru status
Users	GET /api/users, PUT /api/users/:id/role, DELETE /api/users/:id	admin

9 Testare și stabilizare

Proiectul include teste de bază pentru backend și frontend.

9.1 Backend

server/src/app.test.ts verifică:

- endpoint-ul GET /api/health;
- validarea unui payload invalid pentru register.

```
it('returns healthy status', async () => {
  const response = await request(app).get('/api/health');
  expect(response.status).toBe(200);
  expect(response.body.status).toBe('ok');
});
```

9.2 Frontend

client/src/pages/HomePage.test.tsx verifică randarea textului principal din pagina Home.

```
render(
  <BrowserRouter>
    <HomePage />
  </BrowserRouter>,
);

expect(screen.getByText(/Oferte bune la electronice si electrocasnice/i))
  .toBeInTheDocument();
```

9.3 Recomandări pentru extinderea testării

Pentru o predare mai solidă, se recomandă adăugarea următoarelor teste:

- backend: login reușit/eșuat, refresh token, CRUD produse admin, coș, checkout, autorizare admin;
- frontend: login form, catalog cu filtre, adăugare în coș, checkout, protecție rută admin;
- integrare manuală: flux complet register -> verify email -> login -> produse -> coș -> checkout -> istoric comenzi.

10 Integrarea cerințelor din syllabus

Tabelul de mai jos arată cum sunt acoperite modulele din Syllabus.pdf în implementarea curentă.

Modul syllabus	Cerință	Acoperire în proiect
Modul 1: Setup și Arhitectură	frontend/backend, TypeScript, Prisma, contract API	client/, server/, database/prisma/, API_ENDPOINTS.md
Modul 2: Modelare DB și Autentificare	tabele, relații, migrări, seed, register/login/JWT/refresh/reset	schema.prisma, migrări, seed.ts, routes/auth.ts
Modul 3: Catalog Produse	listare, filtrare, sortare, paginare, home/listă/detalii	routes/products.ts, ProductsPage.tsx, ProductDetailsPage.tsx, HomePage.tsx
Modul 4: Coș și Checkout	add/remove/update, checkout simulat, persistare comandă	routes/cart.ts, routes/orders.ts, CartPage.tsx, CheckoutPage.tsx
Modul 5: Admin Panel	roluri, protecție endpoint-uri, CRUD produse/categorii	requireAdmin, AdminProductsPage, AdminCategoriesPage, rute admin
Modul 6: Testare și Stabilizare	teste backend și frontend	app.test.ts, HomePage.test.tsx, scripturi npm de test

10.1 Cerințe minime din syllabus

Cerință minimă	Status	Observații
Register și login funcțional	Implementat	Include verificare email, JWT și refresh token
Listare și detalii produse	Implementat	Catalog cu căutare, filtre, sortare; detalii prin slug
Coș de cumpărături	Implementat	Add/update/delete cu verificare stoc
Istoric comenzi	Implementat	Utilizatorul vede propriile comenzi, admin vede toate
Admin CRUD produse	Implementat	Include validări și protecție rol admin

10.2 Organizarea echipei conform syllabus

Conform syllabus-ului, responsabilitățile sunt împărțite astfel:

- **Barbul Laurențiu:** frontend, UI/UX, integrare API, design bază de date.
- **Oprița Mario:** backend, autentificare și securitate, logică business, testare API.

Această împărțire este reflectată natural în structura proiectului: frontend-ul și integrarea API se află în client/, iar backend-ul, securitatea și logica business în server/.

11 Indicații centralizate pentru imagini

Pentru forma finală de predare, documentul poate fi completat cu următoarele capturi:

1. Screenshot pagina principală: header, brand, căutare, oferte sau produse reprezentative.
2. Screenshot pagina produse: filtre, sortare și carduri de produse.
3. Diagramă ERD: toate tabelele și relațiile bazei de date.
4. Screenshot formular login: câmpuri email/parolă și eventual mesaj de validare.
5. Screenshot coș cumpărături: produse, cantități, total și buton checkout.
6. Screenshot checkout: formular adresă livrare și buton finalizare.
7. Screenshot istoric comenzi: comandă cu status, total și produse.
8. Screenshot Admin Produse: formularul de produs și tabelul de administrare.
9. Screenshot Admin Comenzi: listă comenzi și control de schimbare status.
10. Screenshot teste rulate în terminal: rezultate pentru `npm --prefix server run test` și `npm --prefix client run test`.

12 Concluzii

Proiectul implementează o aplicație e-commerce full stack coerentă, cu separare clară între frontend, backend și bază de date. Arhitectura respectă cerințele din syllabus: React pentru interfață, Node.js/Express pentru API, PostgreSQL pentru persistență și Prisma pentru modelare și migrări.

Punctele forte ale implementării sunt autentificarea JWT cu refresh token, validarea consecventă cu Zod, tranzacțiile Prisma pentru checkout, protecția rutelor admin și integrarea frontend-backend prin React Query și Axios. Pentru îmbunătățiri viitoare, cele mai importante direcții sunt extinderea testelor automate, persistarea sesiunii în browser, rafinarea UX-ului pentru mesaje de succes/eroare și adăugarea unei diagrame ERD finale în documentația predată.